

Shiv Nadar University Chennai

Hemanth Kumar R

24011101035

B.Tech AI&DS 'A'

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

The complete implementation of the Perceptron algorithm developed for this experiment is available in the GitHub repository below. The repository contains the Jupyter Notebook, source code, dataset, and all generated results.

Repository Link

<https://github.com/AI-MANIAC007/Hands-On-Deep-Learning>

7. Experimental Results

The Banknote Authentication dataset was successfully loaded using the Pandas library. The dataset consists of four numerical input features (Variance, Skewness, Kurtosis, and Entropy) and one target variable (Class).

First Five Samples

The first five records of the dataset are shown in Table 7.1.

Table 7.1: First Five Samples of the Dataset

Index	Variance	Skewness	Kurtosis	Entropy	Class
0	3.62160	8.66610	-2.80730	-0.44699	0
1	4.54590	8.16740	-2.45860	-1.46210	0
2	3.86600	-2.63830	1.92420	0.10645	0
3	3.45660	9.52280	-4.01120	-3.59440	0
4	0.32924	-4.45520	4.57180	-0.98880	0

Dataset Dimensions

The dataset contains **1372 samples** and **5 columns**, as shown in Table 7.2.

Table 7.2: Dataset Dimensions

Rows	Columns
1372	5

Dataset Information

The dataset information is summarized in Table 7.3.

Table 7.3: Dataset Information

Feature	Non-Null Count	Data Type
Variance	1372	float64
Skewness	1372	float64
Kurtosis	1372	float64
Entropy	1372	float64
Class	1372	int64

Missing Value Analysis

The dataset was examined for missing values using the `isnull().sum()` function. The results are presented in Table 7.4.

Table 7.4: Missing Value Analysis

Feature	Missing Values
Variance	0
Skewness	0
Curtosis	0
Entropy	0
Class	0

No missing values were found in any feature of the dataset.

Descriptive Statistics

The statistical summary of the dataset is presented in Table 7.5.

Table 7.5: Descriptive Statistics of the Dataset

Statistic	Variance	Skewness	Curtosis	Entropy	Class
Count	1372.000	1372.000	1372.000	1372.000	1372.000
Mean	0.433735	1.922353	1.397627	-1.191657	0.444606
Standard Deviation	2.842763	5.869047	4.310030	2.101013	0.497103
Minimum	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50% (Median)	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
Maximum	6.824800	12.951600	17.927400	2.449500	1.000000

The dataset contains 1372 observations with four numerical input features and one binary target variable. No missing values were observed, and all features are numerical, making the dataset suitable for preprocessing and Perceptron-based binary classification.

8. Generated Plots with Interpretation

Feature Histograms

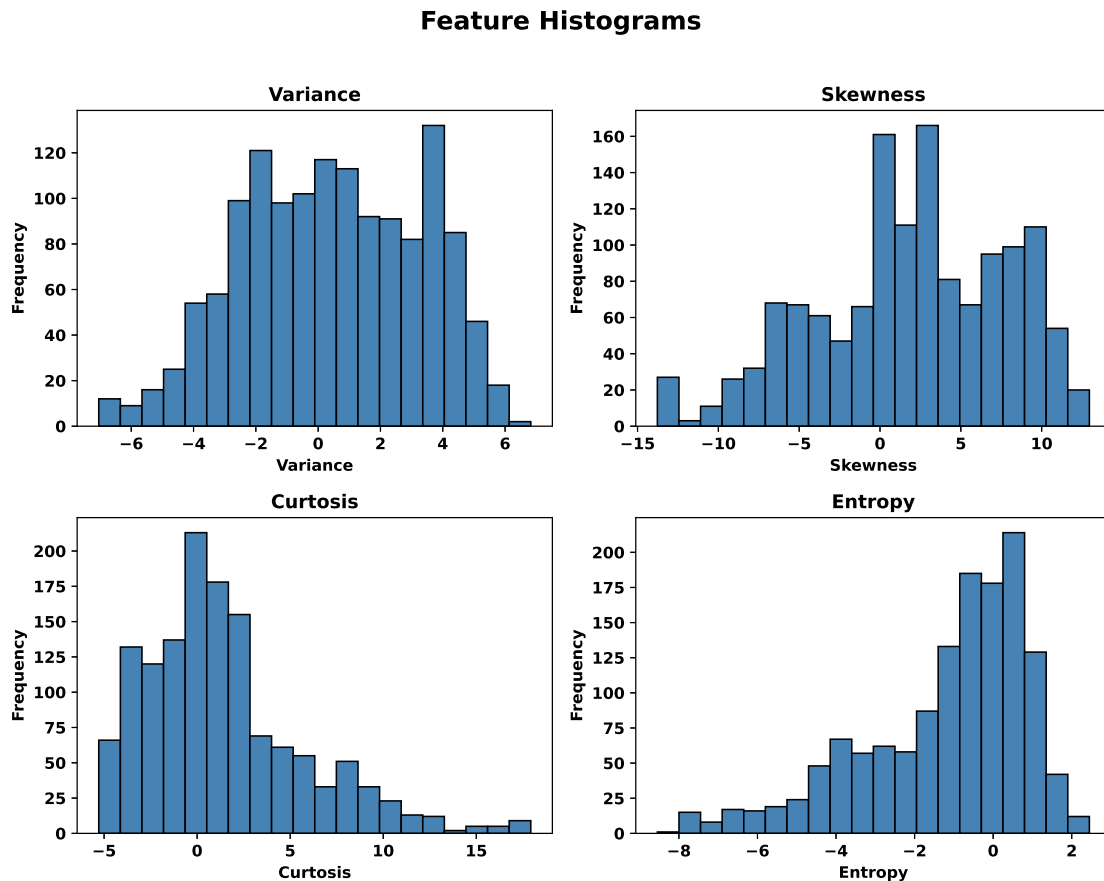


Figure 1: Feature Histograms of the Banknote Authentication Dataset

Variance and Skewness are distributed on negative and positive sides, indicating a wide range of diversity in textures and shapes in the dataset. Kurtosis has a distribution skewed to the right with a long positive tail. Entropy has mainly a distribution around negative to almost zero values, with a few extreme observations. These variations indicate the existence of different feature values in the dataset, which can help the Perceptron to learn meaningful decision boundaries to classify banknotes.

Correlation Heatmap

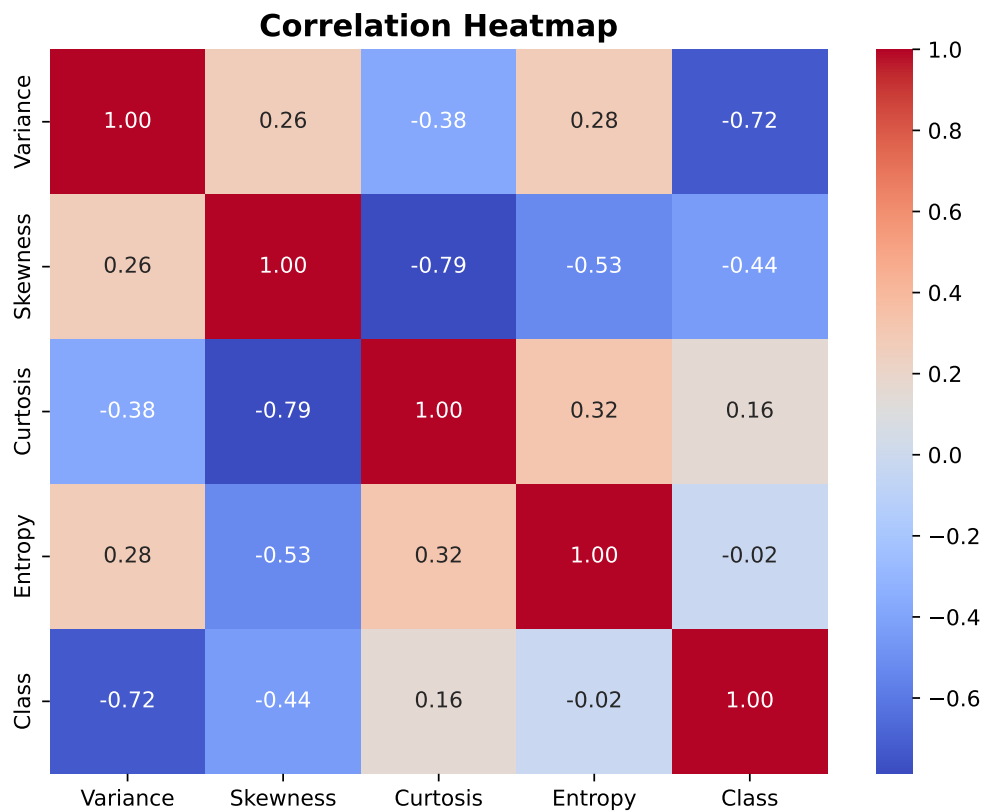


Figure 2: Correlation Heatmap of Dataset Features

The correlation heatmap shows that the most related feature with the target class is Variance (-0.72). This means that the Variance is the most important feature to distinguish between genuine and forged banknotes. Skewness and Kurtosis are strongly negatively correlated (-0.79) which suggests that they are capturing related information. Entropy has very weak correlation with the class (-0.02). Overall, the presence of both strong and weak correlations suggests that each feature plays a different role in the classification task.

Scatter Plot Matrix

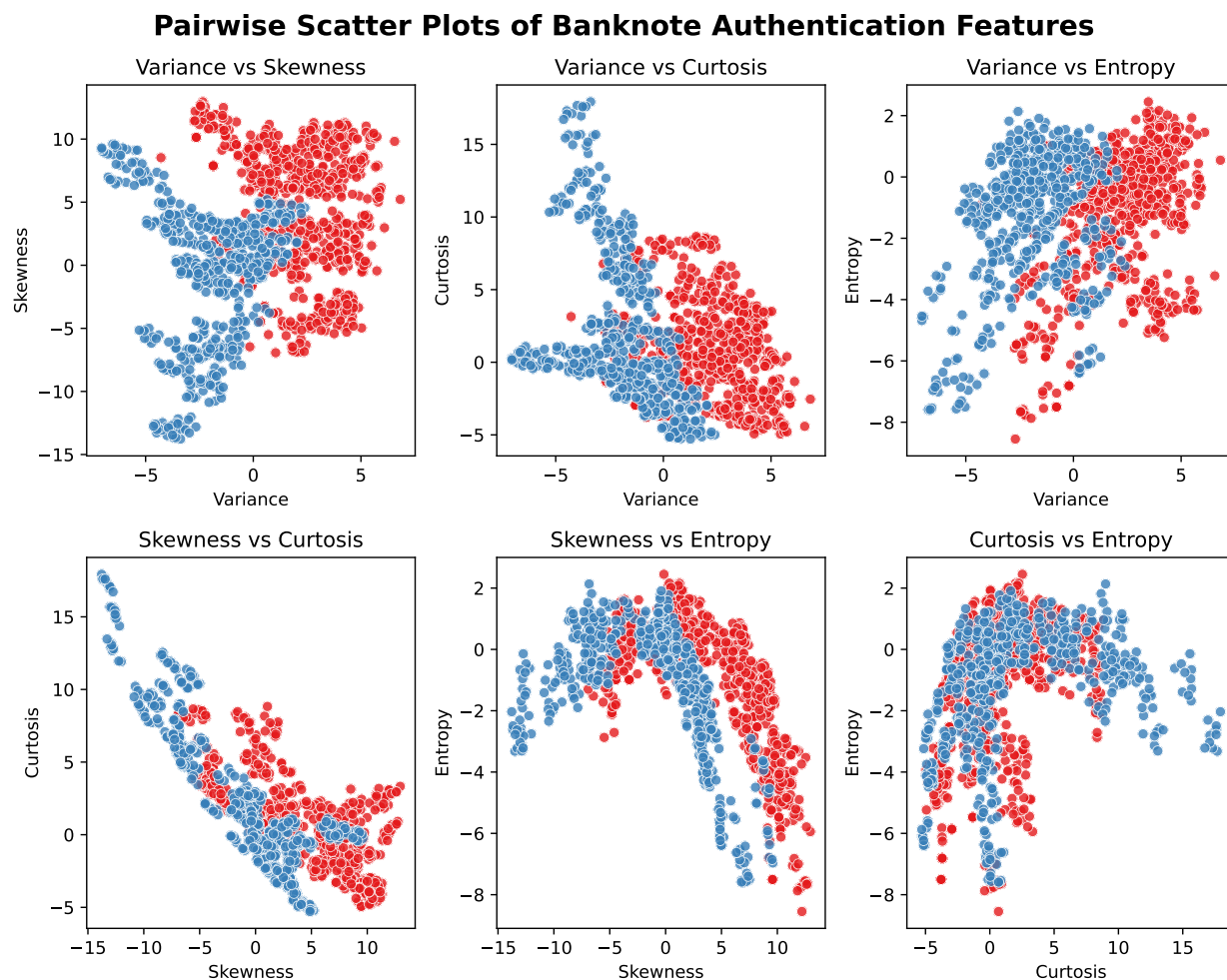


Figure 3: Scatter Plot Matrix Showing Pairwise Feature Relationships

The scatter plot matrix shows the pairwise relationships between the four features, with the two classes shown in different colours. Some feature pairs like Variance versus Skewness and Variance versus Entropy show noticeable class separation, indicating the effectiveness of these features in differentiating genuine and forged banknotes. There are some feature pairs that exhibit non-linear relations and partial overlap between the classes, suggesting that a single feature cannot be used to classify perfectly. But when multiple features are combined, the discrimination is improved and the Perceptron can learn a more accurate decision boundary.

Box Plots

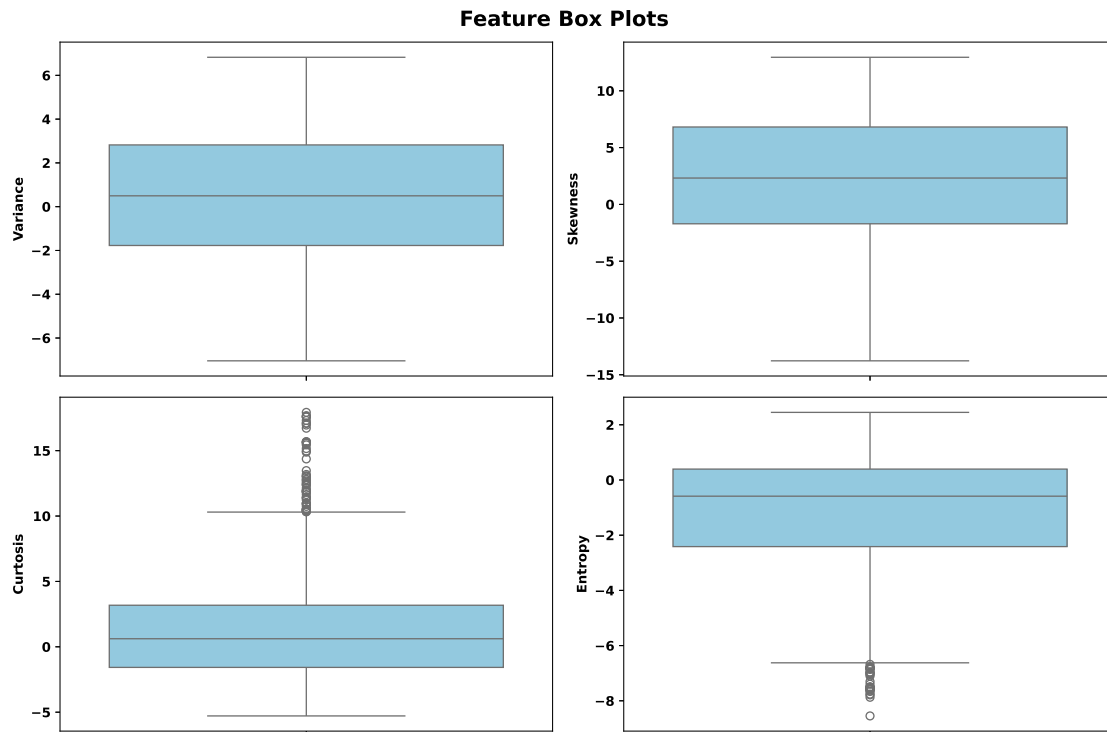


Figure 4: Box Plots of Numerical Features

The box plots summarise the spread and distribution of each numeric feature and whether outliers are present. Variance and Skewness have some balanced distribution in comparison with moderate variation while Kurtosis has some high value outliers and Entropy has some extreme low value observations. In general, the features are in different ranges and variability, which means that each feature provides different information to the dataset.

Training Error versus Epoch



Figure 5: Training Error versus Epoch

The training error decreases sharply during the initial epochs, with the number of misclassified samples dropping from around 170 to around 20, indicating that the Perceptron quickly learns the underlying patterns in the dataset. After this rapid improvement, the error stabilizes with only minor fluctuations, showing that the model has largely converged.

Weight Evolution

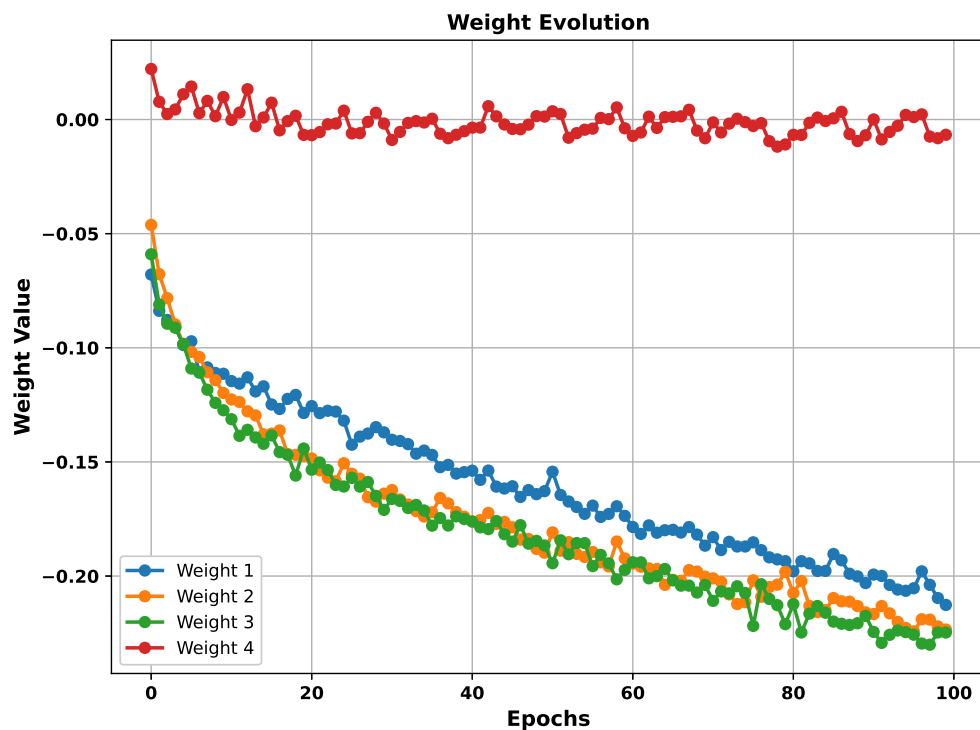


Figure 6: Evolution of Weights During Training

The weight evolution plot shows how the Perceptron updates its feature weights during training to minimize classification errors. Weights 1, 2, and 3 gradually decrease and converge to stable values, while Weight 4 remains close to zero with only small fluctuations, indicating that it has a comparatively smaller influence on the final decision boundary. Overall, the stabilization of all weights after several epochs implies that the Perceptron has converged to an optimal set of parameters.

Bias Evolution

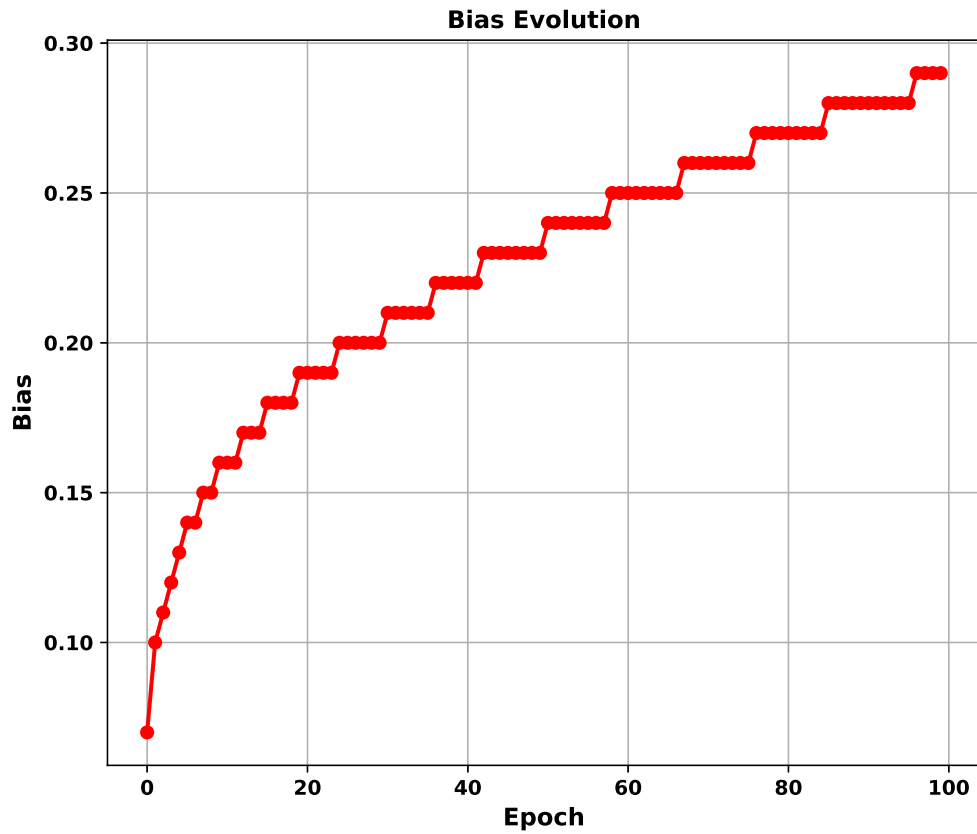


Figure 7: Evolution of Bias During Training

The bias evolution plot shows that the bias increases rapidly during the initial training epochs and then grows more gradually before stabilizing around 0.29. This trend indicates that the Perceptron continuously adjusts the decision threshold to reduce classification errors as learning progresses.

Learning Rate Comparison

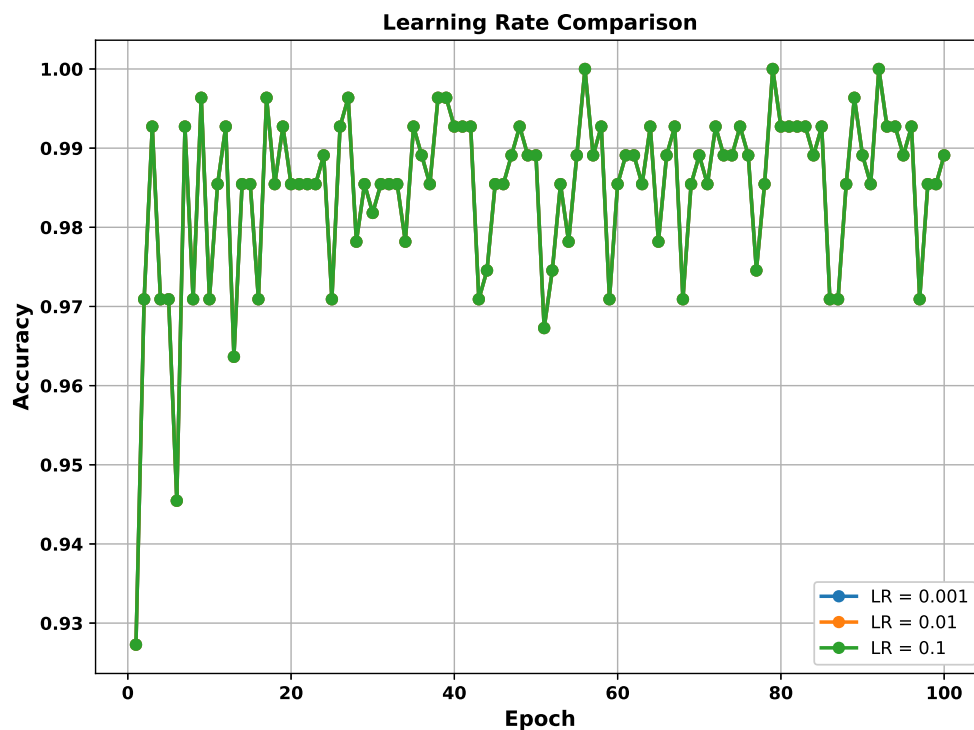


Figure 8: Comparison of Model Performance for Different Learning Rates

The learning rate comparison indicates that all three learning rates (0.001, 0.01, and 0.1) produce nearly identical accuracy throughout training, with the model consistently achieving around 98–99% accuracy. Although the higher learning rate exhibits slightly larger fluctuations in the early epochs, all three converge to a similar performance by the end of training.

Confusion Matrix

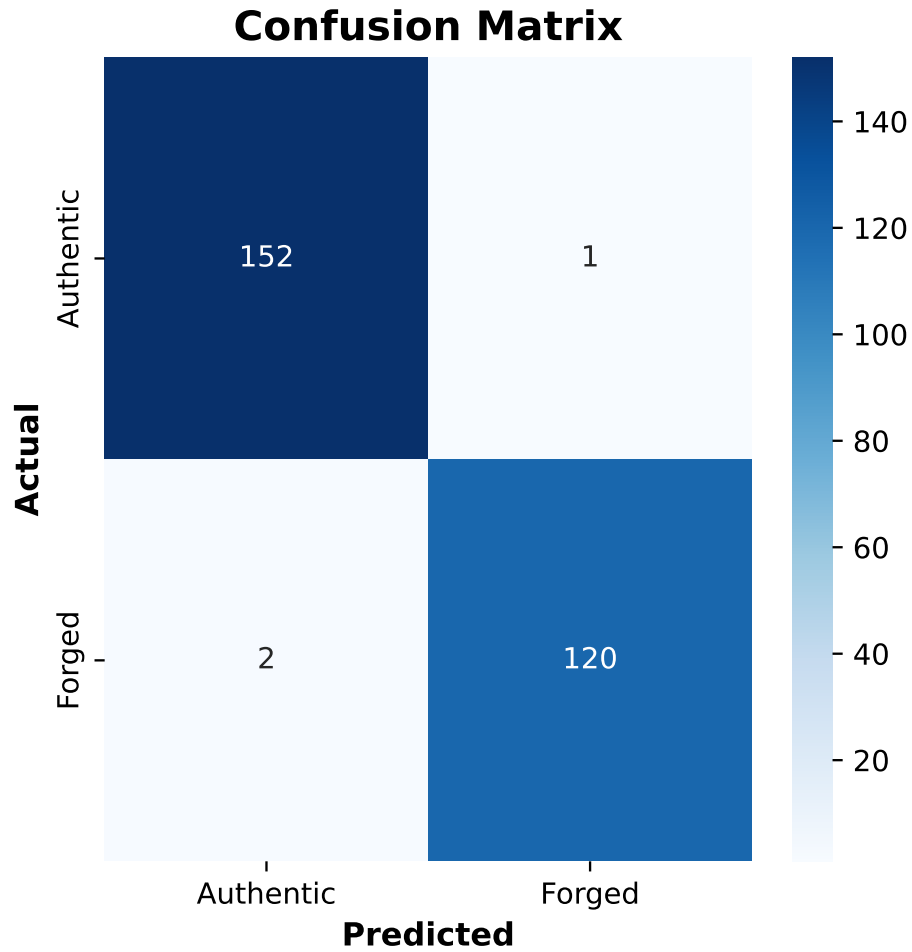


Figure 9: Confusion Matrix of the Perceptron Classifier

The confusion matrix shows that the Perceptron correctly classified 152 authentic and 120 forged banknotes, while misclassifying only 1 authentic banknote as forged and 2 forged banknotes as authentic. The very small number of misclassifications demonstrates that the model is highly effective at distinguishing between the two classes.

Decision Boundary

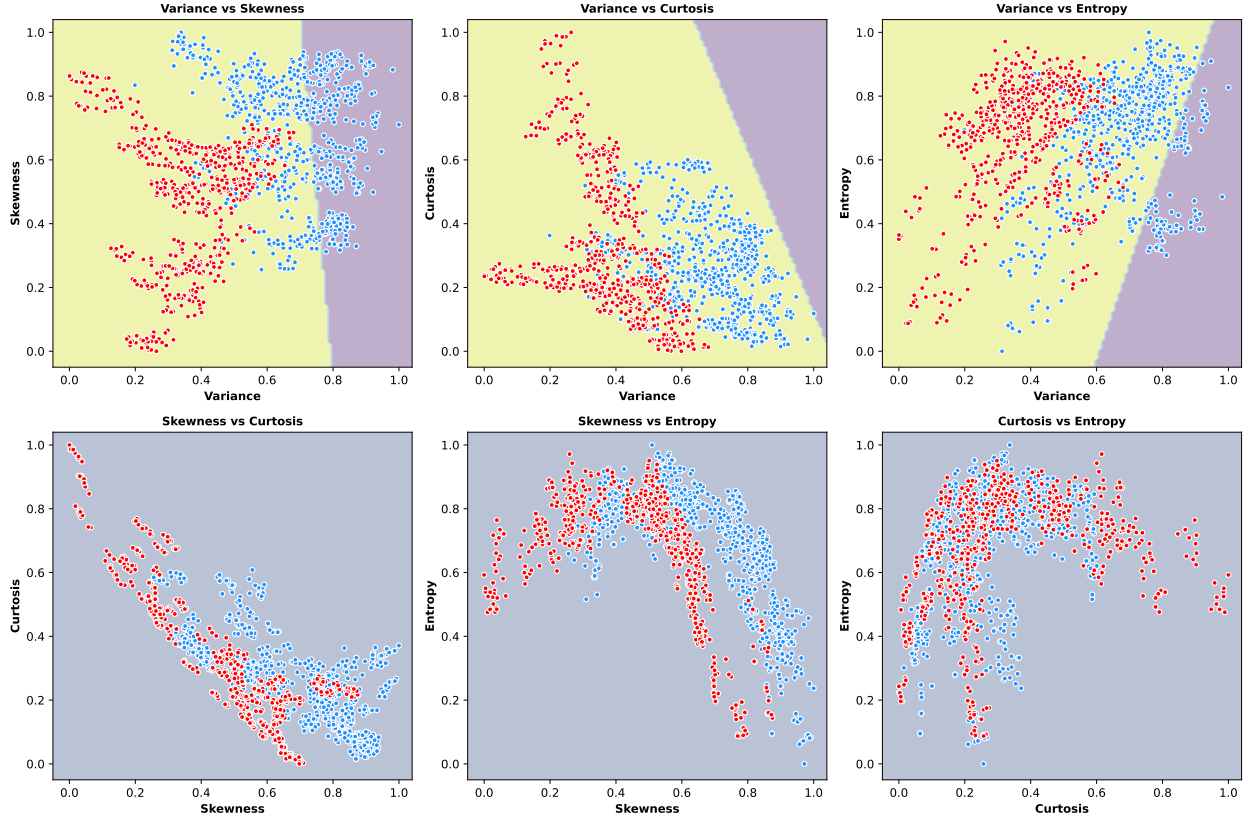


Figure 10: Decision Boundaries for Different Feature Pairs

The decision boundary plots illustrate how the Perceptron separates the two classes using different pairs of features. Clear linear separation is observed for feature combinations such as Variance vs. Skewness, Variance vs. Kurtosis, and Variance vs. Entropy, where the decision boundary effectively divides most authentic and forged banknotes with only a small overlap. In contrast, feature pairs such as Skewness vs. Kurtosis, Skewness vs. Entropy, and Kurtosis vs. Entropy exhibit considerable overlap between the two classes, making them less effective for linear separation. This indicates that Variance is the most discriminative feature, and combining it with other features enables the Perceptron to learn a more accurate decision boundary and achieve high classification performance.

Additional Tasks

Comparison of Step and Sigmoid Activation Functions

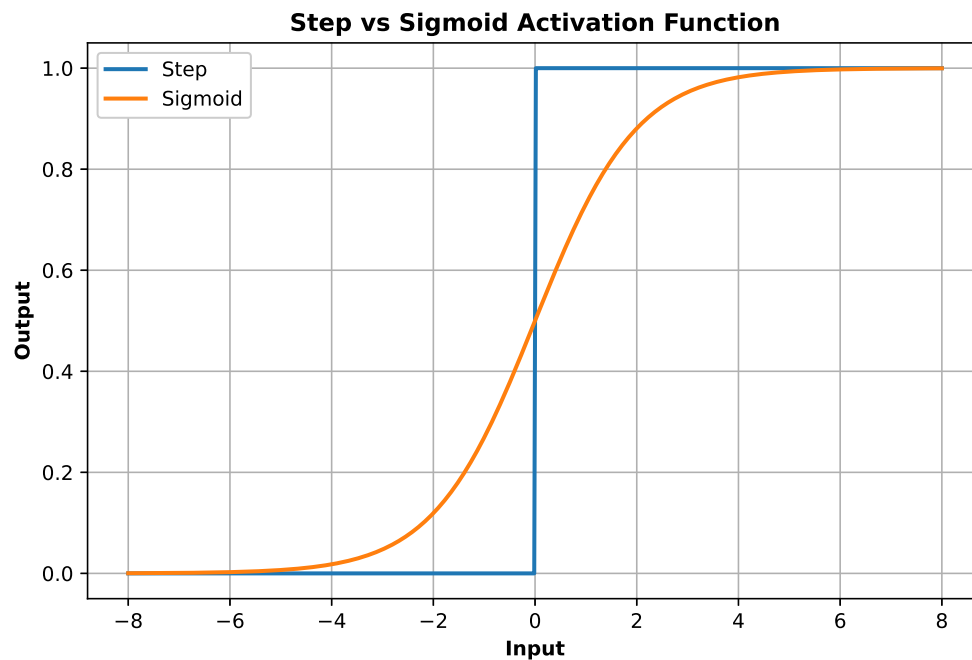


Figure 11: Comparison of Step and Sigmoid Activation Functions

The Step activation function produces only binary outputs (0 or 1), making it suitable for simple Perceptron-based classification. In contrast, the Sigmoid activation function generates smooth outputs between 0 and 1, allowing predictions to be interpreted as probabilities. As shown in the graph, the Step function changes abruptly at the threshold, whereas the Sigmoid function provides a gradual transition.

The Step activation function is not used in modern deep learning because it is non-differentiable and has zero gradient, preventing weight updates through backpropagation. The Sigmoid function is differentiable and supports gradient-based optimization, making it more suitable for training neural networks, although modern architectures often prefer ReLU for improved convergence.

Comparison with Scikit-learn's Perceptron

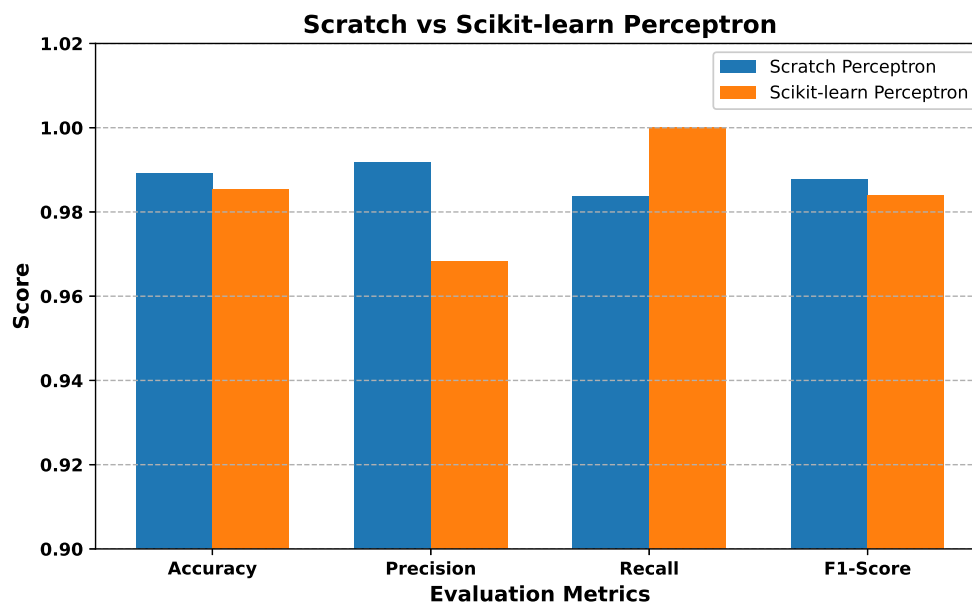


Figure 12: Performance Comparison between Scratch Implementation and Scikit-learn Perceptron

The performance comparison shows that the scratch implementation and Scikit-learn's Perceptron achieve nearly identical results across all evaluation metrics. The scratch model performs slightly better in accuracy, precision, and F1-score, while the Scikit-learn implementation achieves a marginally higher recall. Overall, the minimal differences indicate that the Perceptron implemented from scratch correctly reproduces the learning behavior of Scikit-learn's optimized implementation, demonstrating the correctness and effectiveness of the custom algorithm.

XOR Problem

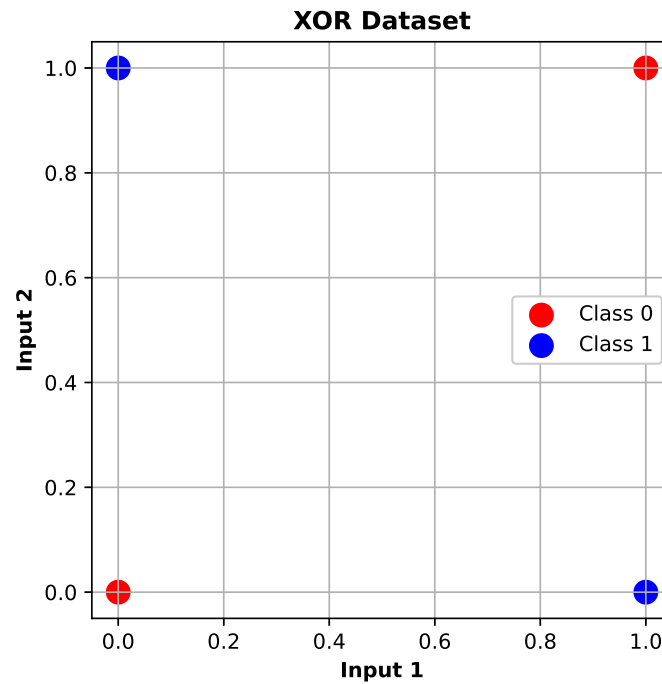


Figure 13: Visualization of the XOR Problem

The XOR dataset shows that the two classes are diagonally opposite each other, making them not linearly separable. As a result, no single straight decision boundary can correctly classify all four data points. Since a single-layer Perceptron can learn only linear decision boundaries, it fails to solve the XOR problem. This limitation led to the development of multi-layer neural networks, where hidden layers and non-linear activation functions enable the model to learn complex, non-linear relationships such as XOR.

Effect of Feature Normalization on Model Convergence

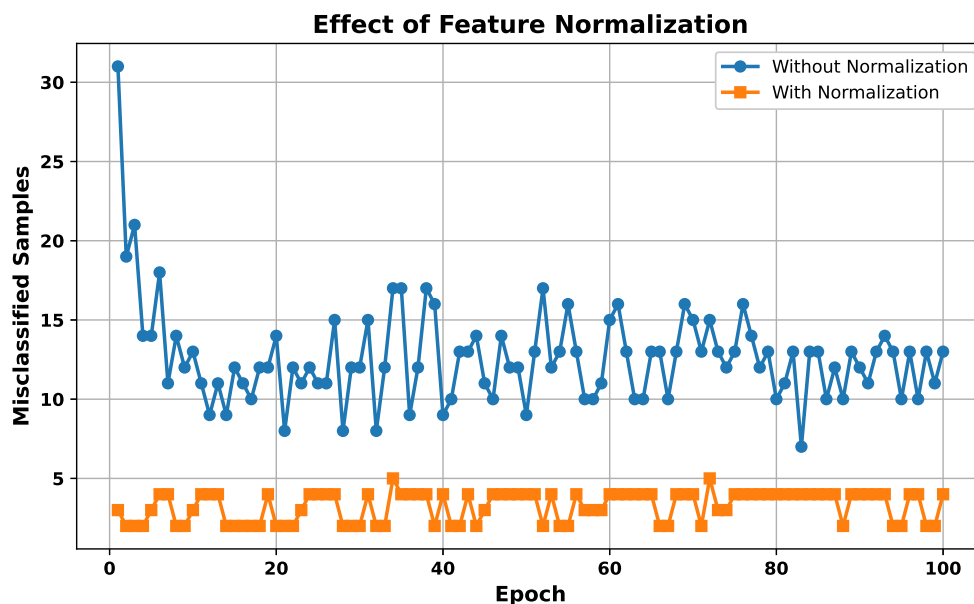


Figure 14: Effect of Feature Normalization on Perceptron Convergence

The graph shows that the Perceptron trained with feature normalization converges much faster and consistently maintains a lower number of misclassified samples than the model trained without normalization. In contrast, the non-normalized model exhibits higher error rates and larger fluctuations throughout training. Overall, feature normalization significantly improves convergence speed and training stability by ensuring that all features contribute equally during weight updates, resulting in more efficient and reliable learning.

9. Performance Tables

Training Summary

Dataset Size	1372 samples (1097 Training, 275 Testing)
Train/Test Split	80:20
Learning Rate	0.01
Maximum Epochs	100
Final Weights	$[-0.0213, -0.0223, -0.0225, -0.0007]$
Final Bias	0.0290
Accuracy	98.91%
Precision	99.17%
Recall	98.36%
F1-score	98.77%

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	171	-0.0679	-0.0462	0.07
2	101	-0.0838	-0.0677	0.10
3	43	-0.0878	-0.0783	0.11
4	43	-0.0908	-0.0898	0.12
5	57	-0.0988	-0.0983	0.13
6	43	-0.0972	-0.1018	0.14
7	30	-0.1108	-0.1041	0.14
8	49	-0.1086	-0.1107	0.15
9	28	-0.1110	-0.1142	0.15
10	43	-0.1114	-0.1198	0.16
11	24	-0.1146	-0.1227	0.16
12	18	-0.1157	-0.1237	0.16
13	33	-0.1130	-0.1278	0.17
14	30	-0.1191	-0.1296	0.17
15	24	-0.1169	-0.1380	0.17

10. Conclusion

This experiment gave a practical insight into how a Single-Layer Perceptron learns binary classification. By implementing the algorithm from scratch, we were able to better understand the role of weights, bias, step activation function and Perceptron learning rule instead of treating the model as a black box. The reduction in training error, the evolution of weights and bias and the final decision boundary clearly indicated that the model was learning gradually from the misclassified samples.

The experimental results show that the Perceptron achieved high classification performance with very few misclassifications, which was also verified by the confusion matrix and the evaluation metrics. The study also discovered that feature normalisation is useful for convergence, making the learning process faster and more stable. The custom implementation was validated with the Scikit-learn Perceptron . Both models produced almost identical performance which confirmed the correctness of the implementation. Finally, in the analysis of the XOR problem, we pointed out an important limitation of the Single-Layer Perceptron: it can only learn linearly separable patterns. This experiment did not only help me to better understand the Perceptron algorithm, but also to understand that why modern neural networks have multiple layers and differentiable activation functions to solve more complex learning problems.

11. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.